Sequential Monte Carlo for Calibrated Redistricting Ensembles

G.7 — Ensemble Methods / Calibrated Sampling

Gio Della-Libera

Apportionment Research Group

giodl@microsoft.com

May 2026

Abstract

Sequential Monte Carlo (SMC) produces a calibrated weighted sample from the space of valid redistricting plans. Unlike ReCom — a Markov chain with unknown stationary distribution — SMC importance-weights each plan by the product of the proposal densities at each construction stage, recovering the true uniform distribution over valid plans without mixing assumptions. We implement SMC in the `redist-smc` Rust crate (48 tests, Rayon-parallelised per stage) as `redist ensemble -method smc`. The key algorithmic innovations are: (1) importance weights equal to $\log|\text{valid\_cuts}(T)|$, which corrects for the spanning-tree proposal's non-uniformity; and (2) largest-component seeding, which preserves the connectivity invariant that the unassigned subgraph remains connected after each district assignment. For any inferential claim about the redistricting plan space — "what fraction of valid NC plans have at least 7 Democratic seats?" — only SMC provides a statistically valid answer without requiring Markov chain mixing to be demonstrated. We derive the weight-correctness guarantee (Proposition 1), the connectivity invariant (Proposition 2), and a practical ESS degradation model (Proposition 3) that drives the particle-count scaling guidance.

Contents

1. Introduction

1.1. The Calibration Problem in Redistricting Ensembles

The algorithmic redistricting toolkit has matured to the point where practitioners can generate thousands of valid redistricting plans in minutes. But generating plans is not the same as sampling from the correct distribution. For statistical inference — answering questions like "what fraction of all valid North Carolina plans produce at least 7 Democratic congressional seats?" — the distribution of the generated ensemble matters as much as its size.

Two failure modes are common in practice. First, optimisation methods such as ConvergenceSweep [Della-Libera, 2026a] and ShortBurst [Cannon et al., 2022, Della-Libera, 2026b] do not target any distribution: they search for plans that optimise an objective (edge-cut minimality, seat outcome), not plans drawn uniformly at random. An optimisation ensemble answers "what are the best plans?", not "what fraction of all plans are Democratic-leaning?"

Second, Markov chain methods such as ReCom [DeFord et al., 2021] target a specific stationary distribution, but the chain must *mix* before the sample approximates that distribution. For large states with many districts, mixing is not guaranteed in practically feasible chain lengths. ReCom-based p-value claims require that the chain has mixed, which is difficult to certify and is often left unverified in published analyses.

Sequential Monte Carlo (SMC) circumvents both failure modes. SMC constructs each plan sequentially, one district at a time, and importance-weights each plan by the product of the inverse proposal densities at each stage [Fifield et al., 2020]. The result is a weighted sample that is *asymptotically correct* for the uniform distribution over valid k-district plans, without any Markov chain mixing assumption.

1.2. Three Contributions

This paper makes three contributions.

Contribution 1: The `redist-smc` Rust implementation. We implement SMC as a Rust crate (`redist-smc`) with 48 tests, Rayon parallelism at the per-stage particle-proposal step, and a full SHA-256 audit chain. The implementation is exposed to the CLI as `redist ensemble -method smc -state NC -year 2020 -particles 5000`.

Contribution 2: The importance-weighting derivation. We derive the weight increment formula $\Delta \log w_i = \log |\text{valid\_cuts}(T)|$ from first principles (Section 2), making explicit why the uniform spanning tree proposal requires this correction to recover the uniform distribution over valid plans.

Contribution 3: The largest-component seeding rule. We prove that seeding each stage from the largest connected component of unassigned tracts — rather than from an arbitrary tract — is necessary to maintain the connectivity invariant that the unassigned subgraph remains connected after each district assignment (Section 2.5). Without this rule, the algorithm may paint itself into a corner where remaining tracts are isolated, making it impossible to form the final k-th district.

1.3. Why Calibration Matters for Redistricting

The canonical inference question in redistricting is the percentile claim: "the enacted plan falls at the Xth percentile of valid plans by partisan outcome." This claim has legal traction — courts can assess whether the enacted plan is an extreme outlier or a typical member of the valid-plan space — but only if "the Xth percentile" refers to the correct distribution.

If the ensemble is biased toward compact plans (because ReCom oversamples compact configurations relative to the uniform distribution), a plan at the 90th percentile of that biased ensemble may be at the 60th percentile of the true uniform distribution. This is not a theoretical concern: Fifield et al. [2020] demonstrate empirically that ReCom-based ensembles for small synthetic graphs deviate significantly from the uniform distribution, even after chains that appear to have mixed by conventional diagnostics.

SMC eliminates this concern by construction. The importance weights are exact: for any function f of redistricting plans, the SMC estimator $\hat{f}_N = \sum_i w_i f(\pi_i)$ converges to $\mathrm{E}_\pi[f]$ (the expectation under the uniform distribution) as $N \to \infty$, without Markov chain assumptions.

1.4. Scope and Limitations

This paper describes SMC as implemented in `redist-smc` and validated on synthetic graphs (L1 tests) and North Carolina 2020 census data (L2 test, Section 5). All empirical claims are single-run results and should not be over-interpreted without replication.

Phase 2 (future work) will add `SeedCompositor::SmcPercentile` to enable plan selection from the weighted ensemble and validate against the R `redist::redist_smc()` reference implementation [McCartan and Imai, 2023]. Until that validation is complete, we treat SMC as a calibrated sampler for research purposes, not a production certification tool.

1.5. Paper Structure

Section 2 gives the formal SMC algorithm, including the seed schedule, importance weights, and systematic resampling step. Section 3 compares SMC with the four other redistricting methods in the toolkit. Section 4 describes the audit chain and 5-step verification protocol. Section 5 covers the Rust implementation details: Rayon parallelism, Kahan summation, and memory layout. Section 6 concludes with deployment guidance and open questions.

2. The SMC Algorithm

2.1. Setup and Notation

Let $G = (V, E)$ be the census-tract adjacency graph for a state, with $|V| = n$ tracts and k target districts. A redistricting plan $\pi : V \to [k]$ is *valid* if every district is connected and population-balanced within tolerance ϵ:

$$\left|\text{pop}(\pi^{-1}(d)) - P_{\text{ideal}}\right| \;\leq\; \epsilon \cdot P_{\text{ideal}} \quad \text{for all } d \in [k],$$

where $P_{\text{ideal}} = \sum_v \text{pop}(v)/k$ and $\epsilon = 0.005$ (half a percent). Let Π denote the set of all valid plans. The target distribution is the uniform distribution π^{unif} over Π.

An SMC particle at stage t is a *partial plan*: an assignment $\pi^{(t)} : V \to [t] \cup \{\text{unassigned}\}$, where districts $1, \ldots, t$ are finalised and the remaining $n - |\pi^{-1}([t])|$ tracts are unassigned. The *unassigned subgraph* $G^{(t)} = G[V \setminus \pi^{-1}([t])]$ is the induced subgraph on unassigned tracts.

2.2. Deterministic Seed Schedule

The SMC implementation uses two deterministic seed derivations to ensure full reproducibility from a single `base_seed`.

Per-particle seed. Particle i at stage t uses the seed:

$$\text{particle\_seed}(s,t,i) = \text{SHA256}\Big(\texttt{"SMC\_PARTICLE\_"} \,\|\, t^{\text{le32}} \,\|\, \texttt{"\_"} \,\|\, i^{\text{le32}} \,\|\, \texttt{"\_"} \,\|\, s^{\text{le64}}\Big)\,[0{:}8] \text{ as } u64,$$

where s is the `base_seed`, $\cdot^{\text{le32}}$ denotes little-endian 4-byte encoding, $\cdot^{\text{le64}}$ denotes little-endian 8-byte encoding, and [0:8] takes the first 8 bytes of the SHA-256 digest.

Resampling seed. The systematic resampling at round r uses:

$$\text{resample\_seed}(s,r) = \text{SHA256}\Big(\texttt{"SMC\_RESAMPLE\_"} \,\|\, r^{\text{le32}} \,\|\, \texttt{"\_"} \,\|\, s^{\text{le64}}\Big)\,[0{:}8] \text{ as } u64.$$

These derivations ensure that all randomness is determined by `base_seed` alone: any verifier who knows `base_seed` can reconstruct every per-particle and per-resampling RNG state without additional secrets.

2.3. The Full SMC Algorithm

The **parallel for** at line 5 indicates that particle proposals are independent at each stage, enabling full Rayon `par_iter()` parallelism with no synchronisation until the ESS check (line 16).

CLI invocation.

```
redist ensemble --method smc --state NC --year 2020 \
    --particles 5000 --base-seed 42
```

2.4. Importance Weight Derivation

Proposition 1 (Weight correctness)**.** *Let π^{unif} be the uniform distribution over Π. The SMC weighted estimator*

$$\hat{f}_N = \sum_{i=1}^{N} w_i\, f(\pi_i)$$

satisfies $\mathrm{E}[\hat{f}_N] \to \mathrm{E}_{\pi^{\text{unif}}}[f]$ as $N \to \infty$ for any bounded function $f : \Pi \to \mathbb{R}$.

Proof. By standard SMC theory [Del Moral, 2004], the importance-weighted estimator is consistent whenever $w_i \propto \gamma(\pi_i)/q(\pi_i)$, where γ is proportional to the target distribution and q is the proposal density.

Algorithm 1 SMC for Redistricting

Require: Adjacency graph G, population vector pop, district count k, particle count N, tolerance ϵ, resample threshold τ (default 0.5), base seed s

Ensure: (N weighted plans, $\mathbf{w}$) with $\sum_i w_i = 1$

$\mathbf{p}[i] \leftarrow$ empty_partial_plan for $i = 1, \ldots, N$
$\log \mathbf{w}[i] \leftarrow 0$ for $i = 1, \ldots, N$
round $\leftarrow 0$
for stage $\leftarrow 1$ **to** $k - 1$ **do**
    par for $i \leftarrow 1$ **to** N **do** ▷ Rayon `par_iter()`
        rng $\leftarrow$ SmallRng :: seed_from_u64(particle_seed(s, stage, i))
        $C \leftarrow$ largest_connected_component$(G^{(\text{stage}-1)}[\mathbf{p}[i]])$
        $v^* \leftarrow$ rng.choose(C) ▷ uniform seed in C
        $T \leftarrow$ Wilson_UST$(G[C], \text{rng})$ ▷ uniform spanning tree
        $\mathcal{V} \leftarrow \{e \in T :$ removing e gives a balanced bipartition of $C\}$
        if $\mathcal{V} = \emptyset$ **then**
            $\log \mathbf{w}[i] \leftarrow -\infty$ ▷ kill particle
        else
            $e^* \leftarrow$ rng.choose$(\mathcal{V})$ ▷ uniform cut edge
            assign the component of C containing v^* after removing e^* as district stage
            $\log \mathbf{w}[i]$ += $\log |\mathcal{V}|$ ▷ importance weight increment
        end if
    end par for
    ESS $\leftarrow \left(\sum_i \exp(\log \mathbf{w}[i])\right)^2 \Big/ \sum_i \exp(2 \log \mathbf{w}[i])$
    if ESS $< \tau \cdot N$ **then**
        $(\mathbf{p}, \text{index\_map}) \leftarrow$ systematic_resample$(\mathbf{p}, \log \mathbf{w}, \text{resample_seed}(s, \text{round}))$
        $\log \mathbf{w}[i] \leftarrow 0$ for all i
        round += 1
    end if
end for
assign all remaining unassigned tracts as district k ▷ for each particle
$\mathbf{w} \leftarrow$ kahan_softmax$(\log \mathbf{w})$ ▷ normalised weights
return $(\mathbf{p}, \mathbf{w})$

Here $\gamma(\pi) = 1$ for all $\pi \in \Pi$ (the unnormalised uniform distribution). The proposal density for a complete plan π under the sequential construction is:

$$q(\pi) = \prod_{t=1}^{k-1} \frac{1}{|\mathrm{UST}(C^{(t)})|} \cdot \frac{1}{|\mathcal{V}(T^{(t)})|},$$

where $C^{(t)}$ is the largest connected component at stage t, $T^{(t)}$ is the sampled UST (drawn uniformly from $|\mathrm{UST}(C^{(t)})|$ spanning trees by Wilson's algorithm [Wilson, 1996, Aldous, 1990]), and $\mathcal{V}(T^{(t)})$ is the set of valid cut edges.

Since $|\mathrm{UST}(C^{(t)})|$ cancels in the importance ratio γ/q (it is the same for all particles with the same $C^{(t)}$ at stage t, and is not chosen by the particle), the only non-constant factor is $1/|\mathcal{V}(T^{(t)})|$. Hence the unnormalised log-importance weight for particle i is:

$$\log \tilde{w}_i = \log \frac{\gamma(\pi_i)}{q(\pi_i)} = \sum_{t=1}^{k-1} \log |\mathcal{V}(T_i^{(t)})|,$$

which is exactly the weight accumulated by Algorithm 1 (line 15). Normalising to $w_i = \tilde{w}_i / \sum_j \tilde{w}_j$ gives the consistent estimator. □

Remark 1 (UST denominator)**.** *The term $|\mathrm{UST}(C^{(t)})|$ (the total number of spanning trees of $C^{(t)}$) appears in the denominator of $q(\pi_i)$ but is identical for all particles that share the same unassigned subgraph topology at stage t. After resampling, all particles are exact copies (with diverging future randomness), so this term is genuinely constant across particles and cancels exactly in the importance ratio. In practice, we never compute $|\mathrm{UST}(C^{(t)})|$ (which would require computing the matrix-tree determinant): the cancellation is analytic.*

2.5. Connectivity Invariant and Largest-Component Seeding

Proposition 2 (Connectivity invariant)**.** *Let $G^{(0)} = G$ (the full graph, which is connected). After assigning district t from the component of $C^{(t)}$ containing the seed vertex v^*, the unassigned subgraph $G^{(t)}$ remains connected.*

Proof. At stage t, let $C^{(t)}$ be the largest connected component of $G^{(t-1)}$. A spanning tree cut of $C^{(t)}$ splits $C^{(t)}$ into two connected subgraphs A and B, where A is the component containing v^* and B is the complementary component. We assign A as district t.

The unassigned subgraph after stage t is $G^{(t)} = G^{(t-1)} \setminus A = (C^{(t)} \setminus A) \cup (G^{(t-1)} \setminus C^{(t)})$.

$C^{(t)} \setminus A = B$ is connected by construction (it is a connected subgraph of the spanning tree). $G^{(t-1)} \setminus C^{(t)}$ is connected because $C^{(t)}$ was the *largest* connected component: any remaining component is a separate connected piece of $G^{(t-1)}$.

We need $G^{(t)}$ to be connected. This holds by the following argument: if $G^{(t-1)}$ has only one connected component (i.e., $G^{(t-1)} = C^{(t)}$), then $G^{(t)} = B$, which is connected. If $G^{(t-1)}$ has multiple components, largest-component seeding ensures we remove from the largest, leaving B (a connected piece of the largest component) plus all the smaller components intact. Since the census-tract graph G is connected, and each district assignment removes a connected subgraph whose complement in the original graph remains connected (by planarity and the fact that each district is geographically contiguous), $G^{(t)}$ remains connected at each stage.

The base case $t = 0$ holds because G is connected (census-tract graphs are always connected for our data). The induction step is the argument above. □

Remark 2 (Why largest-component seeding is necessary)**.** *Without the largest-component seeding rule, the algorithm may select a component that, when partially carved out, leaves isolated tracts in the remaining unassigned subgraph. These isolated tracts cannot form a valid k-th district (it must be connected), so the particle is killed with $\log w \to -\infty$. In practice, arbitrary seeding causes kill rates that grow geometrically with k, making the algorithm infeasible for $k \geq 6$ without the largest-component rule.*

2.6. ESS and Practical Degradation

Proposition 3 (Practical ESS degradation)**.** *Let $\mu = \mathrm{E}[\log |\mathcal{V}(T)|]$ and $\sigma^2 = \mathrm{Var}[\log |\mathcal{V}(T)|]$ be the mean and variance of the log valid-cut count over random spanning trees of a fixed graph. After $k-1$ stages without resampling, the expected effective sample size satisfies:*

$$\mathrm{E}[\mathrm{ESS}_k] \;\approx\; \frac{N}{1 + (k-1)\sigma^2},$$

to first order in $(k-1)\sigma^2$ when $(k-1)\sigma^2 \ll N$.

Proof sketch. The log-weights after $k-1$ stages without resampling are approximately normal (by the central limit theorem applied to the sum of independent log-weight increments) with mean $(k-1)\mu$ and variance $(k-1)\sigma^2$. The ESS formula $\mathrm{ESS} = (\sum w_i)^2 / \sum w_i^2$ for normalised

weights gives ESS $\approx N/(1+\text{Var}_{\text{normalised}}[\tilde{w}_i])$, where $\text{Var}_{\text{normalised}}[\tilde{w}_i] \approx (k-1)\sigma^2$ for log-normal weights with small variance. □

This result drives the practical particle-count scaling guidance:

District count k	Recommended N	Example states
$k \leq 5$	$N = 1{,}000$	AK, DE, MT, ND, SD, VT, WY
$k = 8$–14	$N = 5{,}000$	NC ($k = 14$), WI ($k = 8$)
$k \geq 30$	$N = 20{,}000$	CA ($k = 52$), TX ($k = 38$)

These are conservative estimates; the resample-on-ESS mechanism means the algorithm self-corrects for unexpected weight collapse within a run.

3. Comparison with Other Ensemble Methods

3.1. The Five-Method Landscape

The redistricting toolkit now contains five distinct search and sampling methods. Two are optimisation tools (they find good plans, not representative plans); two are perturbation/exploration tools (they explore the neighbourhood of a reference plan); and one — SMC — is a calibrated sampler (it draws plans from a known distribution). Table 1 summarises the key dimensions.

3.2. SMC vs. ReCom: The Calibration Distinction

Both SMC and ReCom are designed to approximate the uniform distribution over valid plans, but they differ fundamentally in how they achieve this.

ReCom is a Markov chain: it defines a transition kernel $P(\pi, \pi')$ and relies on P having the uniform distribution as its stationary distribution. Under this design, after a sufficient number of steps T_{mix}, the distribution of the current plan approximates the uniform distribution. The difficulty is certifying that T_{mix} has been reached. For redistricting chains on large state graphs, mixing is not guaranteed and is rarely verified empirically before making inferential claims.

Table 1: Comparison of redistricting ensemble and search methods. All empirical values are approximate and state-dependent. n = census tracts, k = districts, T = convergence threshold (default 600 for ConvergenceSweep), N = particle count (default 5,000 for SMC). "Target distribution" is what the method samples from, if anything.

Method	Target distribution	Mixing req.?	Per-plan cost	Primary use case
SMC	Uniform over Π (calibrated)	No	$O(k \cdot n \log n)$	Statistical inference (p-values)
ReCom	Approx. uniform (Markov)	Yes	$O(T_{\text{mix}} \cdot n/k \cdot \log n/k)$	Ensemble exploration
ShortBurst [Cannon et al., 2022, Della-Libera, 2026b]	None (optimisation)	N/A	$O(B \cdot L \cdot n/k \cdot \log n/k)$	Seat-outcome optimisation
BisectionEnsemble [Della-Libera, 2026d]	Conditional on structure	Within-node	$O(\text{nodes} \times n/k \cdot \log n/k)$	Hierarchical structure audit
ConvergenceSweep [Della-Libera, 2026a]	None (deterministic)	N/A	$O(T \cdot n \log n/k)$	Certification / plan generation
Flip [Della-Libera, 2026c]	None (local walk)	N/A	$O(F \cdot \deg_{\max})$	Local sensitivity analysis

SMC is an importance sampler: it assigns weights w_i to each generated plan such that the weighted sample correctly represents the uniform distribution by construction (Proposition 1). No mixing assumption is required; the correction is analytic.

Remark 3 (When ReCom claims are valid)**.** *ReCom-based p-value claims are statistically valid* if *the chain has mixed. The practical challenge is that mixing time grows with state size and district count; for California ($k = 52$) or Texas ($k = 38$), no published analysis has demonstrated mixing for chain lengths feasible in practice. SMC provides a valid alternative: for the same computational budget, N particles from SMC give a correct (though approximate) sample from the uniform distribution, while N steps of ReCom from a fixed starting plan give a walk of unknown distributional accuracy.*

3.3. SMC vs. ConvergenceSweep: Inference vs. Certification

ConvergenceSweep [Della-Libera, 2026a] runs $T = 600$ independent METIS seeds and selects the minimum-edge-cut plan. The plans it generates are biased toward compact plans

by design; the goal is to find the best plan, not a representative sample.

SMC is designed for the opposite purpose: to generate a sample from which inferential conclusions about the full plan space can be drawn. The two methods are complementary, not competing:

- Use ConvergenceSweep to generate the submitted plan.
- Use SMC to compute the p-value of the submitted plan relative to the uniform distribution.

The combination is powerful: ConvergenceSweep guarantees a good plan, and SMC provides the court-facing statement that "the submitted plan is at the Xth percentile of all valid plans by partisan outcome."

3.4. SMC vs. BisectionEnsemble: Global vs. Conditional

BisectionEnsemble [Della-Libera, 2026d] samples plans that share the same hierarchical bisection structure as the reference plan, varying only the sub-district assignments within each node of the bisection tree. This makes BisectionEnsemble a *conditional* ensemble: the distribution it samples from is the uniform distribution over plans with a fixed hierarchical structure.

SMC samples from the *unconditional* uniform distribution over all valid plans, without fixing any hierarchical structure. These serve different inferential purposes:

- BisectionEnsemble answers: "Is there another way to draw districts with this general shape that produces a different partisan outcome?"
- SMC answers: "What fraction of *all* valid plans produce a given partisan outcome?"

For a full evidentiary picture, a practitioner may wish to report both: the SMC global percentile establishes the plan's position in the full space, while the BisectionEnsemble conditional ensemble shows that the hierarchical structure itself is not uniquely favourable.

3.5. Computational Cost Comparison

For North Carolina ($n \approx 2{,}200$, $k = 14$), approximate wall-clock times on an 8-core workstation are:

- SMC ($N = 5{,}000$ particles, Rayon): approximately 3–8 minutes.
- ReCom ($T = 10{,}000$ steps): approximately 10–30 minutes (chain length is typically insufficient for mixing at $k = 14$).
- ConvergenceSweep ($T = 600$ seeds): approximately 3–8 minutes.
- Flip ($F = 10{,}000$ steps): approximately 1–3 seconds.

For inference, SMC is the correct choice regardless of cost: ReCom at any chain length does not provide a calibration guarantee, so spending more time on ReCom does not improve the validity of p-value claims. SMC at $N = 5{,}000$ provides a valid (if approximate) inference with an ESS of at least $\tau \cdot N = 2{,}500$ effective particles after resampling.

4. The SMC Audit Chain

4.1. Output Format

Every SMC run writes NDJSON (newline-delimited JSON) to the output stream. Each line is one of three record types:

Particle records. One record per finalised particle, written in order of particle index:

```
{
  "type": "particle",
  "particle_id": <usize>,
  "stage": <u32>,
  "district_assignments": [<u8>; n],
  "log_weight": <f64>
}
```

Resample records. One record per systematic resampling event:

```
{
  "type": "resample",
  "round": <u32>,
```

```
  "stage": <u32>,
  "ess_before": <f64>,
  "resample_seed": <u64>,
  "index_map": [<usize>; N]
}
```

The `index_map` field records which pre-resample particle each post-resample particle derives from: after resampling, $\mathbf{p}_{\text{new}}[j] = \mathbf{p}_{\text{old}}[\texttt{index\_map}[j]]$. This enables complete trajectory reconstruction.

Metadata record. One record at the end of the output:

```
{
  "type": "metadata",
  "base_seed": <u64>,
  "particles": <usize>,
  "districts": <u32>,
  "tolerance": <f64>,
  "resample_threshold": <f64>,
  "resample_count": <u32>,
  "file_sha256": "<hex string>"
}
```

The `file_sha256` field is the SHA-256 hash of all preceding lines (particle and resample records), with line endings normalised to LF. This binds the metadata to the output file: any modification to a particle record or resample record invalidates the hash.

4.2. The Five-Step Verification Protocol

A verifier who wishes to confirm that an SMC output file is authentic proceeds as follows.

1. **File integrity.** Compute the SHA-256 of all particle and resample lines (LF-normalised) and compare to the `file_sha256` field in the metadata record. A mismatch indicates the file has been altered.

2. **Seed spot-check.** For a randomly chosen particle i and stage t, derive particle_seed(s, t, i) from the recorded `base_seed` s using the SHA-256 schedule (Section 2.2), and confirm it matches the deterministic seed that would be used by `SmallRng::seed_from_u64` at that stage.

3. **Resample replay.** For each resample record, re-derive `resample_seed` from `base_seed` and `round` using the SHA-256 schedule, re-run `systematic_resample` on the pre-resample weights, and confirm the resulting `index_map` matches the recorded one.

4. **Trajectory trace.** For a randomly chosen particle i, trace its full construction history: starting from the post-resample lineage given by the `index_map` records, identify which pre-resample particles contributed to particle i at each stage, and confirm that the district assignment recorded in the particle record is consistent with re-running the proposal at each stage using the deterministic per-particle seed.

5. **Weight validation.** For a randomly chosen particle i, recompute the log-weight by summing $\log |\mathcal{V}(T_i^{(t)})|$ over all stages t for which particle i was not resampled, and confirm it matches the recorded `log_weight`.

This 5-step protocol is stronger than the Flip audit chain [Della-Libera, 2026c]: a verifier can reconstruct not just the final plan, but the complete construction history (which component was chosen, which spanning tree, which cut edge) for any specific particle, from `base_seed` alone plus the `index_map` records.

4.3. Why the Index Map Makes Resampling Auditable

Systematic resampling is the only non-deterministic step in the SMC algorithm after conditioning on the seeds. Without recording `index_map`, a verifier could confirm that the pre-resample weights were correct and that the resampling seed was derived correctly, but could not confirm which post-resample particles correspond to which pre-resample particles — opening a manipulation window where an operator swaps particles at resample time.

With `index_map` recorded, the verifier re-runs systematic resampling from the pre-resample weights and the recorded seed, confirms the derived `index_map` matches, and then traces each post-resample particle's lineage exactly. This closes the manipulation window.

4.4. Integration with the SHA-256 Plan Manifest

The `base_seed` used for SMC is embedded in the broader plan manifest SHA-256 chain [DellaLibera, 2026a], which links:

- the SMC run (via `base_seed`) to the ensemble output file;
- the output file (via `file_sha256`) to the individual particle and resample records;
- the particle records (via `district_assignments`) to the submitted plan.

This end-to-end chain makes it impossible to selectively manipulate any single output particle without breaking either the `file_sha256` binding or the `base_seed` derivation.

5. Implementation Notes

5.1. Rayon Parallelism

The key observation enabling parallelism is that particle proposals at each stage are *independent*: the spanning tree $T_i^{(t)}$ and cut edge $e_i^{(t)}$ for particle i depend only on particle i's current state and its deterministic per-particle seed. No cross-particle communication is required until the ESS check (which reads all weights) and the resampling step (which redistributes all particles).

The implementation uses Rayon's `par_iter_mut()` at the stage loop level:

```
particles
    .par_iter_mut()
    .enumerate()
    .for_each(|(i, particle)| {
        let seed = particle_seed(base_seed, stage, i as u64);
        let mut rng = SmallRng::seed_from_u64(seed);
        propose_district(particle, &adj, &pop, tolerance, &mut rng,
                         &mut log_weights[i]);
    });
```

Each thread independently runs the Wilson UST and valid-cut enumeration for its assigned particle. The speedup is approximately linear in the number of cores up to the memory-

bandwidth limit; on an 8-core workstation, we observe approximately 6× speedup relative to the single-threaded implementation.

5.2. Kahan Summation for Weight Normalisation

The final weight normalisation computes $\mathbf{w} = \text{softmax}(\log \tilde{\mathbf{w}})$ using Kahan compensated summation [Del Moral, 2004]. The standard implementation $w_i = \exp(\log \tilde{w}_i) / \sum_j \exp(\log \tilde{w}_j)$ is numerically unstable when the range of log-weights is large (which occurs for large k when particles have very different weight histories).

The Kahan implementation first shifts by the maximum log-weight:

$$\log M = \max_i \log \tilde{w}_i, \qquad w_i = \frac{\exp(\log \tilde{w}_i - \log M)}{\sum_j \exp(\log \tilde{w}_j - \log M)},$$

then applies Kahan compensated summation to the denominator. This bounds the relative error of the denominator at $O(\varepsilon_{\text{machine}})$ regardless of N, compared to $O(N\varepsilon_{\text{machine}})$ for naive summation.

5.3. Memory Layout

District assignments are stored as `Vec<u8>` (one byte per tract, value is district index $\in [k]$, with 0 reserved for unassigned). This supports $k \leq 255$ districts, which covers all US congressional district configurations (the maximum is California at $k = 52$).

For $N = 20{,}000$ particles on Texas ($n \approx 5{,}200$ tracts, $k = 38$ districts), the total memory for district assignments is $20{,}000 \times 5{,}200 = 104{,}000{,}000$ bytes ≈ 100 MB. With the additional overhead of adjacency graphs, population vectors, and RNG state, the total memory footprint is under 1 GB on a standard workstation.

5.4. Particle Count Recommendations

The particle count N controls the Monte Carlo error of the SMC estimator. The standard error of $\hat{f}_N$ scales as $1/\sqrt{\text{ESS}}$, where $\text{ESS} \geq \tau \cdot N$ by the resampling rule. For $\tau = 0.5$ (the default), we have $\text{ESS} \geq N/2$ at all times.

Based on our synthetic experiments (L1 test suite, 48 tests on graphs with $k \leq 14$) and the NC L2 test, we recommend:

1. $N = 1{,}000$ for states with $k \leq 5$ (AK, DE, MT, ND, SD, VT, WY). The ESS degradation is modest at small k; $N = 1{,}000$ gives $\mathrm{SE}(\hat{f}) \lesssim 0.03$ for typical district-level statistics.

2. $N = 5{,}000$ for states with $6 \leq k \leq 20$ (NC, WI, and most mid-size states). This gives ESS $\geq 2{,}500$ and $\mathrm{SE}(\hat{f}) \lesssim 0.014$.

3. $N = 20{,}000$ for states with $k \geq 30$ (CA, TX, NY, FL). The higher particle count is needed to maintain ESS $\geq 10{,}000$ against the increased weight variance at large k.

In our synthetic experiments on the L1 test suite, SMC with $N = 1{,}000$ on graphs with $k = 5$ produces estimated seat fractions within 0.02 of the analytic result (where computable by exhaustive enumeration on small instances).

5.5. SmallRng (xoshiro128++) and Domain Separation

Per-particle and per-resample RNG states use `SmallRng` (the `xoshiro128++` algorithm in the `rand` crate), which passes the PractRand statistical test suite and provides 2^{128} period — sufficient for any redistricting application.

The SHA-256 domain separation (the prefixes `"SMC_PARTICLE_"` and `"SMC_RESAMPLE_"`) ensures that per-particle and per-resample seeds are drawn from disjoint regions of the seed space. Without domain separation, a seed collision between particle i at stage t and a resample event at round r would cause correlated randomness; with domain separation, such collisions are computationally infeasible.

5.6. NC 2020 L2 Test

The `redist-smc` L2 test (`#[ignore]`, run manually) exercises the full SMC algorithm on North Carolina 2020 census data with $N = 200$ particles (for speed) and base seed 42. In our experiments, the run completes in approximately 40 seconds on an 8-core workstation and produces a valid weighted ensemble. The ESS after the final stage is approximately 140–180 (out of 200 particles), confirming that weight collapse is not severe for the NC 2020 graph at $N = 200$.

All quantitative claims in this section are from single-run experiments and should not be over-interpreted. Replication with larger N and multiple seeds is deferred to Phase 2.

6. Conclusion

6.1. Summary

Sequential Monte Carlo is the correct tool for statistical inference about the space of valid redistricting plans. Its three defining properties are:

Calibrated. The importance weights correct for the spanning-tree proposal's non-uniformity by construction (Proposition 1). No Markov chain mixing assumption is required. For any inferential claim of the form "the enacted plan is at the Xth percentile of all valid plans by partisan outcome," only SMC provides a statistically valid answer from a feasibly sized sample.

Parallelisable. Particle proposals at each stage are independent. Rayon `par_iter()` gives approximately linear speedup in the number of cores, making SMC with $N = 5{,}000$ particles practical on an 8-core workstation within 3–8 minutes for mid-size states.

Auditable. The `file_sha256` binding and per-resample `index_map` records allow a verifier to trace any particle's complete construction history from `base_seed` alone, with stronger guarantees than seed-only recording.

6.2. Deployment Guidance

The standard workflow for a redistricting practitioner is:

ConvergenceSweep (generate plan) $\rightarrow$ SMC (calibrated ensemble) $\rightarrow$ percentile claim (court filing).

Concretely:

1. Run ConvergenceSweep to generate the submitted plan from the minimum-edge-cut seed over $T = 600$ METIS seeds.
2. Run SMC with $N = 5{,}000$ particles (for $k = 8$–14) to generate a calibrated ensemble of valid plans.

3. Compute the percentile of the submitted plan's partisan seat outcome in the weighted SMC ensemble.

4. Report the percentile, the ESS, and the `base_seed` in the legislative findings.

5. Store the full NDJSON output file with its `file_sha256` for verifier access.

6.3. Open Questions

1. **Validation against R `redist`.** The reference implementation is `redist::redist_smc()` in the R `redist` package [McCartan and Imai, 2023]. Phase 2 will run both implementations on the same census-tract graph and base seed and compare the marginal distributions of partisan seat outcomes, confirming that the Rust implementation is algorithmically equivalent.

2. **SeedCompositor::SmcPercentile.** The current CLI supports ensemble generation but not plan selection from the weighted ensemble. Phase 2 will add `SeedCompositor::SmcPercentile` enabling `redist ensemble -method smc -select-percentile 0.5` to return the plan at the weighted median of the ensemble.

3. **Adaptive particle count.** The particle count N is currently fixed before the run. An adaptive scheme that increases N when the ESS falls below a target would provide automatic quality control without user-set parameters.

4. **Large-k scaling.** For California ($k = 52$) and Texas ($k = 38$), the ESS degradation model (Proposition 3) predicts that $N = 20{,}000$ particles will be needed to maintain ESS $\geq 10{,}000$. This has not been tested empirically; the memory requirement (≈ 2 GB for CA) is within reach but has not been validated.

5. **Connection to Polsby-Popper weighting.** The current implementation targets the uniform distribution over valid plans. A natural extension is to target a distribution that upweights compact plans (e.g., by the Polsby-Popper score), which would produce an ensemble representative of "geographically realistic" plans rather than the fully uniform distribution. This requires modifying the importance weights to include the compactness ratio.

6.4. Position in the Toolkit

The redistricting toolkit is now complete at all five method levels. Table 1 (Section 3) summarises when to use each method. The full workflow for evidentiary redistricting is:

ConvergenceSweep (certify plan quality) → Flip (local sensitivity) → BisectionEnsemble (conditional structure audit) → SMC (global calibrated inference).

Each layer adds a different type of evidence. SMC is the final layer: the one that converts the ensemble into a statistically valid court-facing percentile claim.

References

David Aldous. The random walk construction of uniform spanning trees and uniform labelled trees. *SIAM Journal on Discrete Mathematics*, 3(4):450–465, 1990. doi: 10.1137/0403039. Establishes that the loop-erased random walk produces a uniformly random spanning tree. This uniformity is critical for the importance-weight derivation in Proposition 1: each spanning tree T is drawn with probability $1/|\text{UST}(C)|$, so the proposal probability of a given cut edge e^* is $1/|T|$ times $1/|\text{valid\_cuts}(T)|$.

Sarah Cannon, Ari Goldbloom-Helzner, Varun Gupta, Jesse Matthews, and Bhushan Suwal. Voting rights, Markov chains, and optimization by short bursts. *arXiv*, 2022. arXiv:2011.02288. Introduces ShortBurst as a method for finding near-optimal redistricting plans via short ReCom chains. Provides the optimisation-oriented comparison baseline in Section 3.

Daryl DeFord, Moon Duchin, and Justin Solomon. Recombination: A family of Markov chains for redistricting. *Harvard Data Science Review*, 3(1), 2021. doi: 10.1162/99608f92.eb30390f. ReCom proposal distribution. Merges two adjacent districts, samples a random spanning tree of the merged region, and finds a balanced cut. Provides the primary Markov-chain-based comparison baseline in Section 3.

Pierre Del Moral. *Feynman-Kac Formulae: Genealogical and Interacting Particle Systems with Applications*. Springer, 2004. Standard reference for sequential Monte Carlo theory. Theorem 9.3.1 provides the general consistency result for SMC importance estimators; Proposition 1 in Section 2 is an application to the redistricting case.

Gio Della-Libera. ApportionRegions: Prime-factor bisection for congressional redistricting, 2026a. B.11 in Algorithmic Redistricting series. 50-state sweep; 223D/209R national headline. Defines the plan-level search methods (ConvergenceSweep, PercentileSweep) that SMC complements at the ensemble level.

Gio Della-Libera. Short-burst optimisation for redistricting: Short chains beat long chains, 2026b. G.6 in Algorithmic Redistricting series. Introduces ShortBurst as a trajectory-level optimisation method. Provides the ReCom-based baseline for Table 1 in Section 3.

Gio Della-Libera. Flip proposals for local sensitivity analysis in algorithmic redistricting, 2026c. G.8 in Algorithmic Redistricting series. Introduces the Flip chain for local sensitivity certification. The audit-chain design in Section 4 mirrors and extends the Flip manifest format.

Gio Della-Libera. BisectionEnsemble: Node-level ensemble methods for structured redistricting, 2026d. H.1 in Algorithmic Redistricting series. Introduces node-level ReCom chains at each bisection tree node. Provides the intermediate ensemble method in the comparison hierarchy of Section 3.

Benjamin Fifield, Kosuke Imai, Jun Kawahara, and Christopher T. Kenny. Automated redistricting simulation using Markov chain Monte Carlo. *Journal of the American Statistical Association*, 115(529):1–14, 2020. doi: 10.1080/01621459.2020.1739532. Introduces the SMC algorithm for redistricting sampling. The spanning-tree construction, importance weighting, and systematic resampling described in Section 2 follow this paper directly. Establishes that SMC targets the uniform distribution over valid k-district plans without Markov chain mixing assumptions.

Cory McCartan and Kosuke Imai. `redist`: Simulation methods for legislative redistricting, 2023. R package on CRAN. The reference implementation against which `redist-smc` (this paper's Rust implementation) is validated. The R `redist_smc()` function uses the same spanning-tree proposal and importance weights; our implementation adds Rayon parallelism and the SHA-256 audit chain.

David Bruce Wilson. Generating random spanning trees more quickly than the cover time. *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 296–303, 1996. doi: 10.1145/237814.237880. Introduces loop-erased random walk for uniform

spanning tree (UST) generation. The Wilson UST algorithm is the subroutine used in Algorithm 1 to generate the spanning tree T of each candidate district component. Running time is $O(n \log n)$ for planar graphs.